

Please find all Pseducode of the attacks at the bottom of the notebook, after references

```
In [ ]: # please read the below important text
```

Important!! for the setting up part(having to install and all), if we **run all** , the collab runtime might get disconnected. So if that happens, just for the **setting up part** only, it is best to run cell by cell to avoid that.!!

```
In [ ]: # end of important text
```

Audioseal

As an overview, **AudioSeal** is the response to address the need to ensure authenticity against the risk of voice cloning, especially with the rise of more, and more natural sounding voices.

AudioSeal is the first audio watermarking technique designed specifically for localized detection of AI-generated speech. The below screenshots are from the original paper.

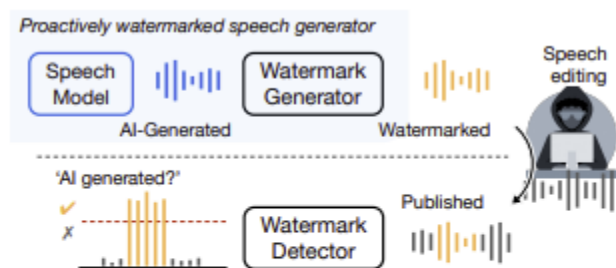


Figure 1. Proactive detection of AI-generated speech. We embed an imperceptible watermark in the audio, which can be used to detect if a speech is AI-generated and identify the model that generated it. It can also precisely pinpoint AI-generated segments in a longer audio with a sample level resolution (1/16k seconds).

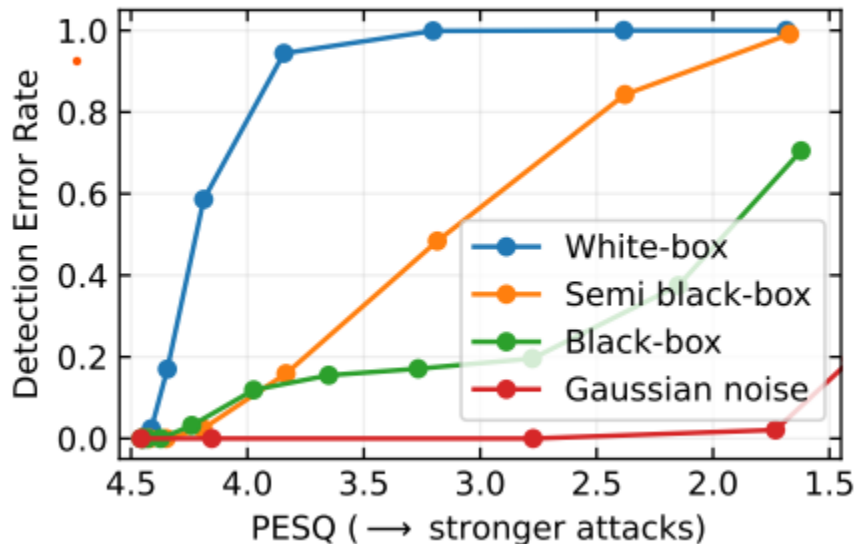
To summarize the above image, AI generated voice will go through the **Watermark Generator** which will essentially embeds a signal into the audio. Later, the Watermark Detector should be able to detect the Watermark.

A major caveat of **AudioSeal** is that it relies on the presence of the

watermark (Roman et al., 2024, p.6).

So if the audio did not go through the **Watermark Generator**, even if its an AI generated voice (that sounds authentic), the model will classify the audio as non-watermarked.

For the sake of our assignment, all test audio we will go through the **Watermark Generator** and to test the effectiveness of our attacks.



Straight off the bat, we can see that White Box attacks are much more superior than the rest. However, in reality, white box attacks are not as easy to come by especially if you do not have full access to the models.

While in this case we do have access, we took the initiative to explore other attacks just to understand the capability of `AudioSeal`.

Consider below to be the chapters of our journey in coming up with both understanding and attacking `AudioSeal`.

Setting up AudioSeal and other dependencies

Before we begin anything, we'll first set up all the necessary things. This part, we'll introduce 3 things:-

- `AudioSeal` - A system that includes both watermark generation and detection for audio files.
- `PESQ` - A metric used to evaluate how much the audio quality changes after applying our attacks.
- `LibriSpeech` - Open source speech dataset that we'll be using throughout this investigation.
- `Spectrogram & Waveform` - Code to display the spectrogram & waveform of the audio.

We also relied on `AudioSeals` [Getting_started.ipynb](#) as a guide as well.

AudioSeal setup

This code helps set up `AudioSeal`, which adds and checks for watermarks in audio files. It loads the tools to create and detect the watermark, and there's a little function to tell us if an audio file is probably watermarked or not.

The `print_result` is just to print the results in a readable way. The threshold we'll use is 0.5, as seen in the screenshot from the paper.

- *Detection:* To determine if the audio is watermarked or not. To achieve this, we use the average detector's output over the entire audio and flag it if the score exceeds a threshold (default: 0.5).

```
In [ ]: !pip install audioseal --quiet
```

```

59.6/59.6 kB 2.0 MB/s eta
0:00:00
Preparing metadata (setup.py) ... done
117.0/117.0 kB 4.6 MB/s eta
0:00:00
Preparing metadata (setup.py) ... done
79.5/79.5 kB 2.1 MB/s eta 0:0
0:00
Building wheel for julius (setup.py) ... done
Building wheel for antlr4-python3-runtime (setup.py) ... done
```

```
In [ ]: from audioseal import AudioSeal
```

```

# loading the watermark generator and detector
watermark_generator = AudioSeal.load_generator("audioseal_wm_16bits")
watermark_detector = AudioSeal.load_detector("audioseal_detector_16bits")
```

```
def print_result(result):
    """
    prints whether the audio is likely watermarked or not based on the re

    if the result is 0.5 or higher, the audio is probably watermarked.
    otherwise, it's likely unwatermarked.

    parameters:
    result (float): the probability that the audio is watermarked.
    """
    # check if the result meets the watermark threshold of 0.5
    if result >= 0.5:
        # print watermark probability and indicate it's likely watermarke
        print(f"Watermark probability: {result*100:.2f}%\nThis is likely
    else:
        # print watermark probability and indicate it's likely unwatermar
        print(f"Watermark probability: {result*100:.2f}%\nThis is likely u
```

```
Downloading: "https://huggingface.co/facebook/audioseal/resolve/main/gene
rator_base.pth" to /root/.cache/audioseal/0f195d476dd87ca1bd7b09e6
100%|██████████| 56.1M/56.1M [00:01<00:00, 42.2MB/s]
Downloading: "https://huggingface.co/facebook/audioseal/resolve/main/dete
ctor_base.pth" to /root/.cache/audioseal/94c8df0b1d5ea8e45af4c884
100%|██████████| 33.1M/33.1M [00:00<00:00, 37.6MB/s]
```

PESQ setup

Note: **PESQ (Perceptual Evaluation of Speech Quality)**, tells us how well a sound retains its quality after being processed or transmitted ([Rix et al., 2001](#))-a-of-Rix-Berends/dd5e786fd6ced91db79105ca289f49816fe17c80). As a thumb rule:-

- 1) **4.0 to 4.5:** Excellent quality, almost no degradation. The sound is very close to the original.
- 2) **3.0 to 4.0:** Good quality, but may have light differences from the original audio.
- 3) **2.0 to 3.0:** Fair quality, noticeable degradation in sound quality, but still understandable.
- 4) **Below 2.0:** Poor quality, significant distortion or noise making the audio difficult to understand.

```
In [ ]: !pip install pesq --quiet
```

```
Preparing metadata (setup.py) ... done
Building wheel for pesq (setup.py) ... done
```

```
In [ ]: !pip install torchmetrics --quiet
```

890.6/890.6 kB 13.4 MB/s eta 0:00:00

LibriSpeech

LibriSpeech is a popular dataset made up of thousands of hours of English speech, often used for speech recognition and audio-related tasks. We're using just a small subset of the audio samples to keep things manageable and fast for testing purposes and its not like we're training anything. And since, we're always forcing the audio to be undetectable in our code, any audio sample from the source we had chosen, will display the same behaviour. Hence why we're only using one as a demo.

```
In [ ]: import os
import torchaudio

# make a directory for the dataset
os.makedirs('LibriSpeech_dataset', exist_ok=True)

# load the 'test-clean' subset of the librispeech dataset
librispeech_test = torchaudio.datasets.LIBRISPEECH(root='LibriSpeech_data')

# grab the first sample from the dataset for demo purpose
selected_samples = [librispeech_test[0]]
```

100%|██████████| 331M/331M [00:13<00:00, 26.5MB/s]

Spectrogram & Waveform

The below code is taken from the **AudioSeal** [github](#) repo. The main function is to display the spectrogram & waveform as well as playing the audio.

```
In [ ]: import IPython.display as ipd
import matplotlib.pyplot as plt
import torch

def plot_waveform_and_spectrogram(waveform, sample_rate, title):
    """
    plots both the waveform and spectrogram of an audio sample.

    parameters:
    waveform (tensor): the audio data (waveform) to be visualized.
    sample_rate (int): the sample rate of the audio.
    title (str): the title for the plot.
```

```

    this function creates a side-by-side view of the waveform and its spectrogram
    """
    # squeeze removes any unnecessary dimensions, detach for no gradients
    waveform = waveform.squeeze().detach().cpu().numpy()

    # determine the number of frames and create a time axis based on the
    num_frames = waveform.shape[-1]
    time_axis = torch.arange(0, num_frames) / sample_rate

    # create a figure with two subplots, one for the waveform and one for
    figure, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 4))

    # plot the waveform on the first subplot
    ax1.plot(time_axis, waveform, linewidth=1)
    ax1.grid(True)

    # plot the spectrogram on the second subplot
    ax2.specgram(waveform, Fs=sample_rate)

    # set the overall title and show the plots
    figure.suptitle(f"{title} - waveform and spectrogram")
    plt.show()

def play_audio(waveform, sample_rate):
    """
    plays the audio directly in a notebook cell.

    parameters:
    waveform (tensor): the audio data (waveform) to be played.
    sample_rate (int): the sample rate of the audio.

    this function handles both mono (1 channel) and stereo (2 channel) audio
    """
    # squeeze any extra dimensions if needed
    if waveform.dim() > 2:
        waveform = waveform.squeeze(0)
    waveform = waveform.detach().cpu().numpy()

    # check how many channels the audio has
    num_channels, *_ = waveform.shape
    # if mono audio, play normally
    if num_channels == 1:
        ipd.display(ipd.Audio(waveform[0], rate=sample_rate))
    # if stereo audio, play both channels
    elif num_channels == 2:
        ipd.display(ipd.Audio((waveform[0], waveform[1]), rate=sample_rate))
    # raise an error if there are more than 2 channels, since they're not supported
    else:
        raise ValueError("waveform with more than 2 channels is not supported")

```

```

In [ ]: # setting seed
        torch.manual_seed(47)

```

Out[]: <torch._C.Generator at 0x7d97f4de5670>

Watermarking the list of audios

Before we proceed, we'll use the loaded audio files from before and watermark them beforehand, to both show the effectiveness of the `AudioSeal`.

```
In [ ]: # list to hold the watermarked audio data and the detection probabilities
watermarked_audio_list = []
watermark_probabilities = []

# loop over the dataset, watermark each sample, and then detect the watermark
for idx, (audio_waveform, sample_rate, label, _, _, _) in enumerate(selected_samples):
    print(f"Watermarking and processing sample {idx + 1}/{len(selected_samples)}")

    # ensure the audio tensor has the right shape
    audio_waveform = audio_waveform.unsqueeze(0)

    # generate and apply the watermark
    watermark = watermark_generator.get_watermark(audio_waveform, sample_rate)
    watermarked_audio = audio_waveform + watermark

    # detect the watermark in the watermarked audio
    result, _ = watermark_detector.detect_watermark(watermarked_audio, sample_rate)

    print(f"Watermark detection result for sample {idx + 1}:")
    print_result(result)

    # add the watermarked audio to the lists
    watermarked_audio_list.append((watermarked_audio, sample_rate))
    watermark_probabilities.append(result)
```

```
Watermarking and processing sample 1/1...
Watermark detection result for sample 1:
Watermark probability: 100.00%
This is likely watermarked
```

Highly recommend running the above cells one by one, to avoid disconnection of runtime ↑↑↑

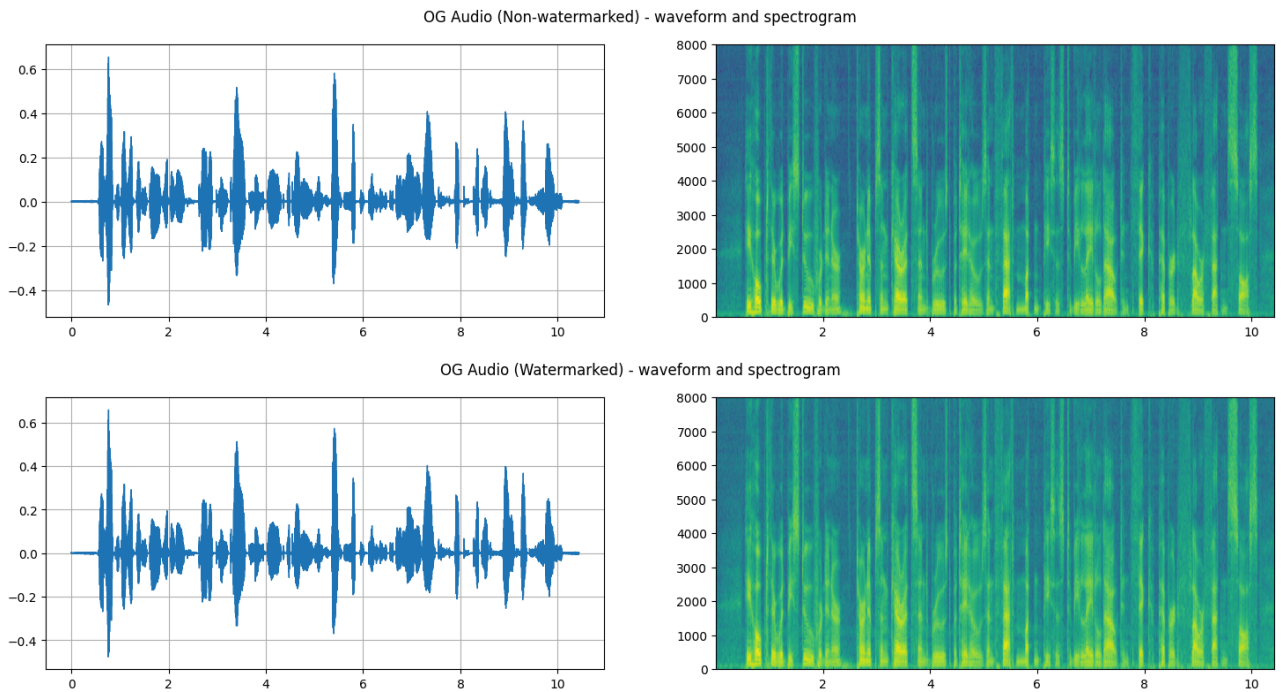
You can start `run cell` and `below` from here, as it shouldn't cause any runtime disconnection ↓↓↓

```
In [ ]: # extracting before watermarking audio
before_watermark_audio = selected_samples[0][0]

# after watermarking
```

```
after_watermark_audio = watermarked_audio_list[0][0]
```

```
In [ ]: plot_waveform_and_spectrogram(before_watermark_audio, sample_rate, 'OG Aud  
plot_waveform_and_spectrogram(after_watermark_audio, sample_rate, 'OG Audi
```



```
In [ ]: from torchmetrics.audio.pesq import PerceptualEvaluationSpeechQuality  
  
# initializeing the pesq metric  
# fs (sample rate) is the frequency of the audio, and wb (wideband) mode  
pesq = PerceptualEvaluationSpeechQuality(fs=sample_rate, mode='wb')  
  
# calculate pesq score for the original audio and watermarked audio  
pesq_score_demo = pesq(before_watermark_audio.squeeze(), after_watermark_  
  
print(f"PESQ Score: {pesq_score_demo:.2f}")
```

PESQ Score: 4.50

```
In [ ]: play_audio(before_watermark_audio, sample_rate)  
play_audio(after_watermark_audio, sample_rate)
```

Your browser does not support the audio element.

Your browser does not support the audio element.

And as claim in the paper, the embedding of the watermark had caused almost no distortion to the audio file. The above plots is just to show what we mean. As we can see, the watermarking is so subtle that no perturbation is caused, yet still able to acheive 100% in watermark detection rate. This efficiently shows `AudioSeals` effectiveness.

Attacks

Querying

All papers will be properly referenced at end of notebook

We were inspired to implement the query attack based on [Carlini and Wagner's](#) work on adversarial examples, which demonstrated how small, imperceptible perturbations (random noise essentially) can lead to significant changes in model behavior. Applying similar principles, our query attack aims to gradually modify the audio in a way that fools the watermark detector while maintaining the audio quality.

This is fundamentally not a complicated attack, but for the sake of investigating, we attempted it to see how `AudioSeal` respond with our own very eyes.

The screenshot below is from the their authors' paper: [Audio Adversarial Examples: Targeted Attacks on Speech-to-Text](#)

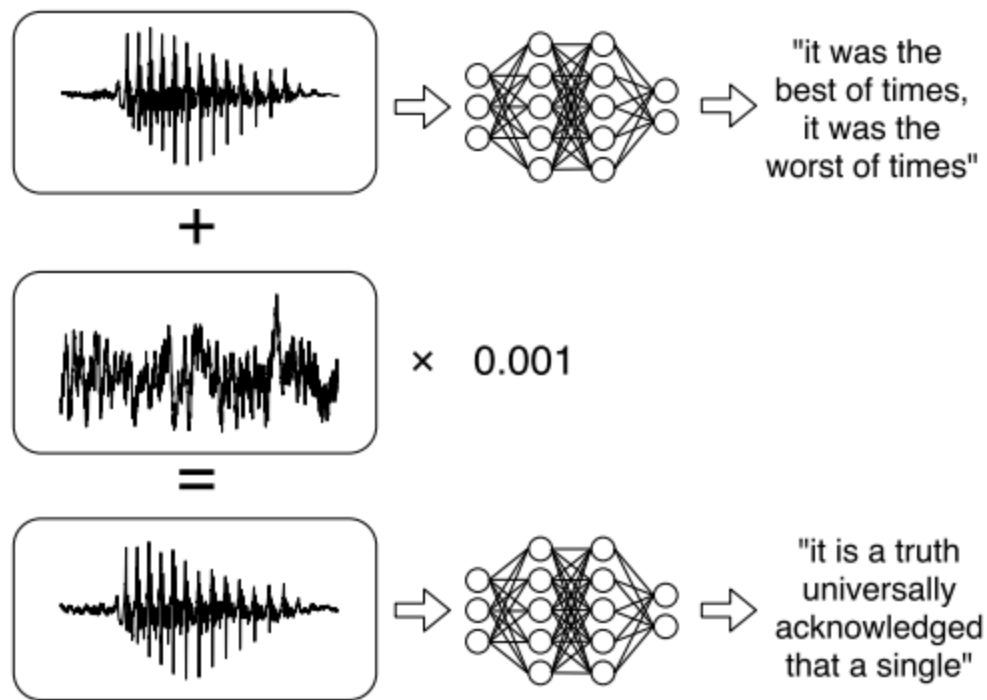


Figure 1. Illustration of our attack: given any waveform, adding a small perturbation makes the result transcribe as any desired target phrase.

In this query attack, we add a small amount of Gaussian noise (random noise) to the audio data in each iteration. The goal is to gradually alter the audio in a way that still sounds imperceptible to human ears but fools the watermark detector.

That being said, we tried to replicate it the above study. In the study the used 0.001 which was quick and effective in removing the watermark (although you can clearly hear the noise in the background which will be presented later) so this may not be as succesful as we had hoped.

```
In [ ]: # to store probability and audio after each query attack
query_audio_list = []
query_probabilities = []

# initialize variables for the perturbation loop according to the above p
perturbation_strength = 0.001
max_iterations = 1000 # maximum number of iterations
upper_threshold = 0.5 # detection threshold

# loop over the watermarked audio files
```

```

for idx, (watermarked_audio, sample_rate) in enumerate(watermarked_audio):
    print(f"Applying query attack on sample {idx + 1}/{len(watermarked_audio)}")

    # clone the watermarked audio to create a perturbed version
    perturbed_audio = watermarked_audio.clone()

    # list to hold the detection probabilities for this sample
    sample_probabilities = []

    # perturbation loop for the current audio sample
    for i in range(max_iterations):
        # apply random perturbation to the audio
        perturbation = torch.randn_like(perturbed_audio) * perturbation_scale
        perturbed_audio += perturbation

        # query the detector
        detection_prob, _ = watermark_detector.detect_watermark(perturbed_audio)

        # print detection probability
        print(f"Sample {idx + 1}, Iteration {i + 1}, Detection Probability: {detection_prob}")

        # check if the watermark is no longer detected to avoid it from going on
        if detection_prob < upper_threshold:
            print(f"Watermark successfully removed or evaded for sample {idx + 1}")
            break

    # add the perturbed audio and its final detection probability to the lists
    query_audio_list.append(perturbed_audio)
    query_probabilities.append(detection_prob)

```

Applying query attack on sample 1/1...

```

Sample 1, Iteration 1, Detection Probability: 100.00%
Sample 1, Iteration 2, Detection Probability: 99.74%
Sample 1, Iteration 3, Detection Probability: 95.73%
Sample 1, Iteration 4, Detection Probability: 94.29%
Sample 1, Iteration 5, Detection Probability: 90.11%
Sample 1, Iteration 6, Detection Probability: 88.71%
Sample 1, Iteration 7, Detection Probability: 83.21%
Sample 1, Iteration 8, Detection Probability: 83.17%
Sample 1, Iteration 9, Detection Probability: 72.47%
Sample 1, Iteration 10, Detection Probability: 72.16%
Sample 1, Iteration 11, Detection Probability: 71.40%
Sample 1, Iteration 12, Detection Probability: 73.45%
Sample 1, Iteration 13, Detection Probability: 59.69%
Sample 1, Iteration 14, Detection Probability: 53.92%
Sample 1, Iteration 15, Detection Probability: 53.19%
Sample 1, Iteration 16, Detection Probability: 60.00%
Sample 1, Iteration 17, Detection Probability: 51.95%
Sample 1, Iteration 18, Detection Probability: 47.12%
Watermark successfully removed or evaded for sample 1 at iteration 18!
Detection Probability: 47.12%

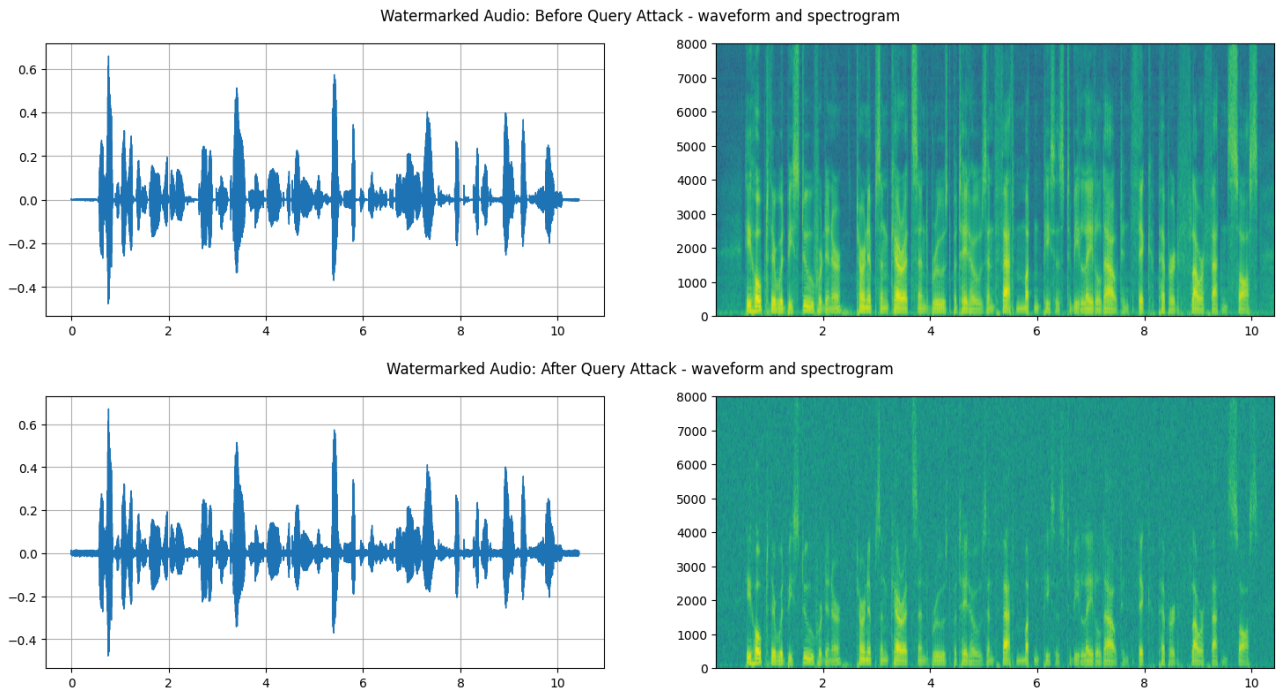
```

```

In [ ]: # sample audio after query attack
sample_query_audio = query_audio_list[0][0]

```

```
In [ ]: plot_waveform_and_spectrogram(after_watermark_audio, sample_rate, 'Waterma
plot_waveform_and_spectrogram(sample_query_audio, sample_rate, 'Watermarke
```



As we can see above, despite the noise added, in terms of the waveform, we can see that there is not any noticeable change. However, in the spectrogram we can clearly see there is some slight alteration to the original watermarked audio. Let's have a look at the PESQ score for this query attack.

```
In [ ]: # calculating pesq score between the watermarked & after query attack
pesq_score_query = pesq(after_watermark_audio.squeeze(), sample_query_aud
print(f"Query --> PESQ Score: {pesq_score_query:.2f}")
```

Query --> PESQ Score: 1.91

```
In [ ]: play_audio(after_watermark_audio, sample_rate)
play_audio(sample_query_audio, sample_rate)
```

Your browser does not support the audio element.
Your browser does not support the audio element.

Checkpoint 1 Evaluation

Here we can see that, this simple method of adding noise to the audio till the detector can't detect the watermark, does work. But in terms of audio quality which is dependent on the run, the PESQ scores can range from 1.6 - 2.3 . Which is fine, this part motivated us in exploring other black-box methods

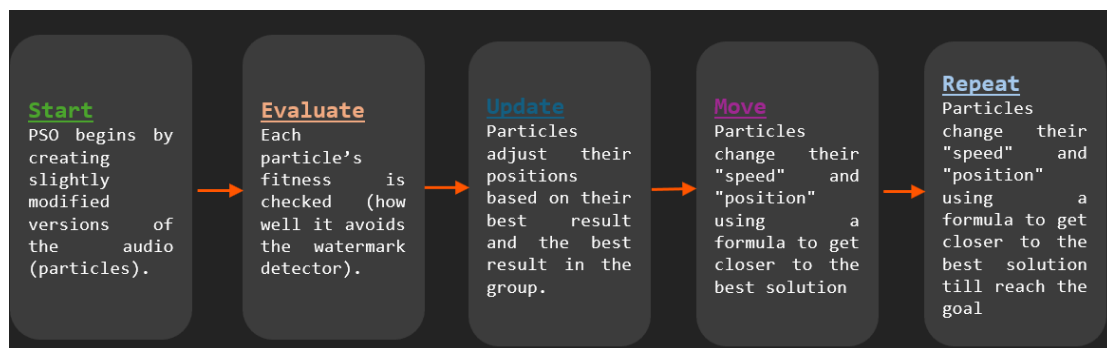
that could hopefully give us consistent results while still evading detection.

PSO

At this stage, we tried re-searching papers on other advanced black box attacks and we randomly stumbled upon a paper called [Black-Box Audio Adversarial Attack Using Particle Swarm Optimization](#). The study was conducted on one-second audio files (containing words like 'NO', 'YES' etc) and utilizes PSO to find the optimum attack that would somehow misclassify the audio by the classifier while also being imperceptible to human ears.

This part is important to note as we will not be conducting the attack on one-second audio but rather audio clips ranging to 10 seconds on average. But for the sake of exploration, we decided to give it a try to see if we're able to remove the watermark in a more intelligent way. Regardless, this approach intrigued us and we would like to implement this, and to see how much it differ than the normal querying method.

a little intro about PSO



Velocity update: $v_i = w \cdot v_i + c_1 \cdot r_1 \cdot (pBest_i - x_i) + c_2 \cdot r_2 \cdot (gBest - x_i)$

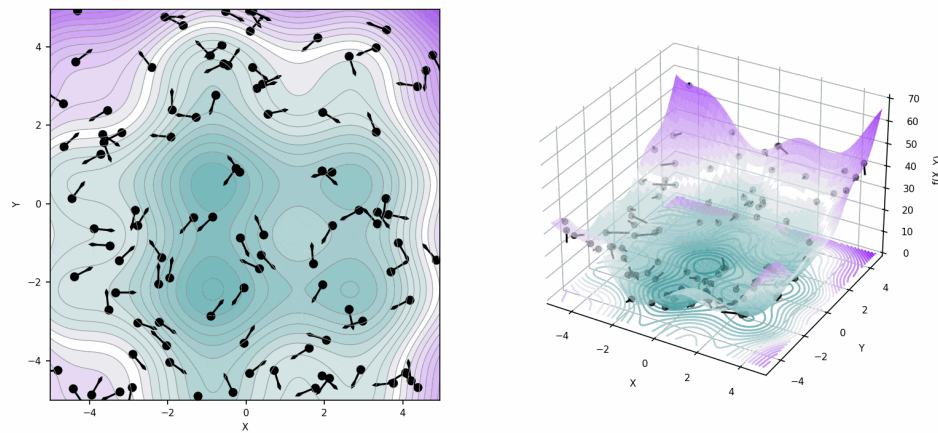
Position update: $x_i = x_i + v_i$

Where:

- w is the inertia weight
- c_1 and c_2 are cognitive and social learning coefficients respectively
- r_1 and r_2 are random numbers (used to introduce randomness and increase exploration).

Below is simply visualization. Source: [Here](#)

[1/100] w:0.800 - c₁:3.500 - c₂:0.500



```
In [ ]: import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

def particle_swarm_audio_watermark(watermarked_audio, detector, sr, swarm_size):
    """
    generate adversarial audio using particle swarm optimization to evade
    while keeping the perturbations small.
    """

    # initialize the swarm with particles (each a modified version of the
    particles = []
    for _ in range(swarm_size):
        # generate some noise the same size as the audio
        noise = torch.randn_like(watermarked_audio) * initial_epsilon
        # each particle starts as the audio + some noise
        particle = watermarked_audio + noise
        particles.append({
            "position": particle, # current position of the particle (th
            "velocity": torch.zeros_like(watermarked_audio), # initial v
            "best_position": particle.clone(),
            "best_fitness": float('inf') # tracks how well it's doing at
        })

    # initialize the global best (the best solution found by the whole sw
    global_best_fitness = float('inf') # set as infinity initially
    global_best_position = None # to be determined later
    fitness_over_time = [] # can store fitness values over time (optiona

    # function to check how well the audio avoids the watermark detector
    def evaluate_fitness(audio):
        # assume the detector returns a probability score of how detectab
        result, _ = detector.detect_watermark(audio, sample_rate=sr)
```

```

    return result

# main loop for particle swarm optimization
for iteration in range(max_iterations):
    # use epsilon decay to slowly reducing noise
    epsilon = initial_epsilon * (1 - iteration / (2 * max_iterations))

    # evaluate each particle and update their best results
    for particle in particles:
        fitness = evaluate_fitness(particle["position"]) # check how

        # update the particle's personal best if it finds a better ve
        if fitness < particle["best_fitness"]:
            particle["best_fitness"] = fitness
            particle["best_position"] = particle["position"].clone()

        # update the global best if this particle does better than al
        if fitness < global_best_fitness:
            global_best_fitness = fitness
            global_best_position = particle["position"].clone()

    # update the velocity and position of each particle
    for particle in particles:
        r1 = torch.rand(1).item() # random factor 1
        r2 = torch.rand(1).item() # random factor 2

        # calculate new velocity based on personal and global best po
        particle["velocity"] = (
            w * particle["velocity"] # previous velocity (inertia)
            + c1 * r1 * (particle["best_position"] - particle["positi
            + c2 * r2 * (global_best_position - particle["position"])
        )

        # update the particle's position (the version of the audio it
        particle["position"] += particle["velocity"]
        # make sure the audio stays within a valid range (-1.0 to 1.0
        particle["position"] = torch.clamp(particle["position"], -1.0

    # terminating condition
    if global_best_fitness < 0.5:
        print(f"Terminating with global best fitness {global_best_fit
        break

# return the best version of the audio found
adversarial_audio = global_best_position
return adversarial_audio

```

```

In [ ]: # list to hold adversarial audio data and detecton probabilities
pso_audio_list = []
pso_probabilities = []

# loop through the watermarked audio list and apply pso on each sample
for idx, (watermarked_audio, sample_rate) in enumerate(watermarked_audio_
    print(f"Running PSO on sample {idx + 1}/{len(watermarked_audio_list)}

```

```

# run the particle swarm optimization to generate adversarial audio
adversarial_audio = particle_swarm_audio_watermark(
    watermarked_audio=watermarked_audio, # the watermarked audio to
    detector=watermark_detector,         # the watermark detector
    sr=sample_rate,                      # sample rate of the audio
    swarm_size=10,                       # number of particles in the
    max_iterations=20,                   # maximum number of iteratio
    initial_epsilon=0.005,               # starting noise level for p
    c1=1, c2=2,                          # cognitive and social compo
    w=0.7,                               # inertia weight that contro
)

# query the detector for the detection probability of the adversarial
detection_prob, _ = watermark_detector.detect_watermark(adversarial_a

# add the adversarial audio
pso_audio_list.append((adversarial_audio, sample_rate))

# add the detection probability of the adversarial audio to the list
pso_probabilities.append(detection_prob)

print(f"Adversarial audio generated for sample {idx + 1} with detecti

```

Running PSO on sample 1/1...

Terminating with global best fitness 0.3849484920501709

Adversarial audio generated for sample 1 with detection probability 38.49%.

```

In [ ]: # extract audio after pso attack
sample_pso_audio = pso_audio_list[0][0]

```

```

In [ ]: # calculating pesq score between the watermarked & after pso attack
pesq_score_pso = pesq(after_watermark_audio.squeeze(), sample_pso_audio.s

print(f"PSO --> PESQ Score: {pesq_score_pso:.2f}")

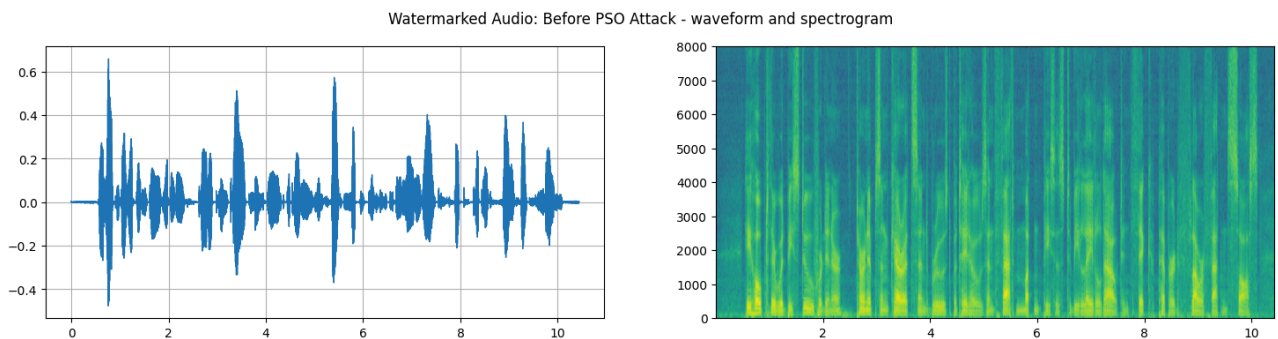
```

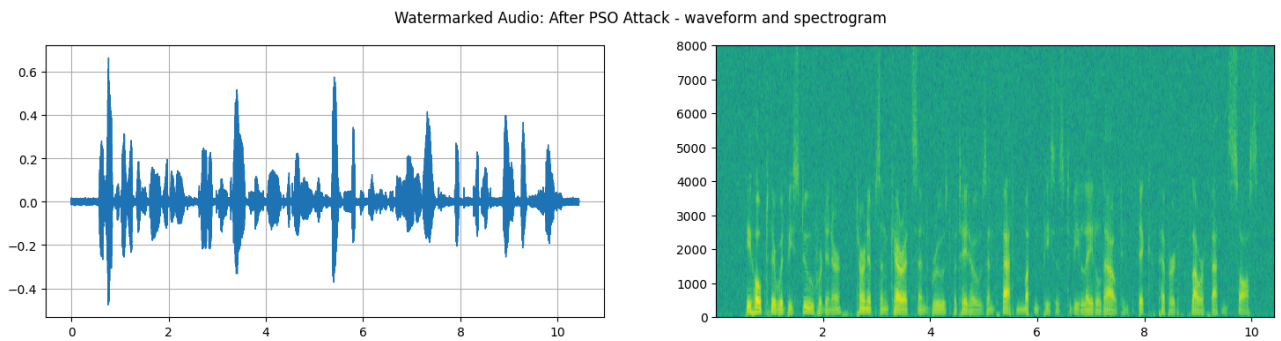
PSO --> PESQ Score: 1.75

```

In [ ]: plot_waveform_and_spectrogram(after_watermark_audio, sample_rate, 'Waterma
plot_waveform_and_spectrogram(sample_pso_audio, sample_rate, 'Watermarked

```





```
In [ ]: play_audio(after_watermark_audio, sample_rate)
        play_audio(sample_pso_audio, sample_rate)
```

Your browser does not support the audio element.
Your browser does not support the audio element.

Checkpoint 2 evaluation

So, as we can see, the **PSO** attack is more structured in the way it attacks, using the group's knowledge/swarm optimization to find the best solution. However, the parameters of the PSO is extremely crucial.

But in a few trials, we did actually find that **depending on the run**, if in the **query** attacks it is able to evade the detection probability earlier in earlier iterations, it may produce a better PESQ score and **(sometimes)** lower detection probability than **PSO** attack. Sometimes, the PESQ attacks for both attacks are more or less the same.

The below screenshot, is when a query attack manages to find an attack early on which may able to evade detection while still maintaining a higher audio quality rather than the PSO attack, and is much simpler, although unreliable.

Example of Query attack where ended in early iteration:-

```
Applying query attack on sample 1/1...
Sample 1, Iteration 1, Detection Probability: 100.00%
Sample 1, Iteration 2, Detection Probability: 98.62%
Sample 1, Iteration 3, Detection Probability: 90.80%
Sample 1, Iteration 4, Detection Probability: 87.67%
Sample 1, Iteration 5, Detection Probability: 81.75%
Sample 1, Iteration 6, Detection Probability: 75.71%
Sample 1, Iteration 7, Detection Probability: 70.06%
Sample 1, Iteration 8, Detection Probability: 67.91%
Sample 1, Iteration 9, Detection Probability: 62.59%
Sample 1, Iteration 10, Detection Probability: 64.37%
Sample 1, Iteration 11, Detection Probability: 59.93%
Sample 1, Iteration 12, Detection Probability: 55.13%
Sample 1, Iteration 13, Detection Probability: 50.85%
Sample 1, Iteration 14, Detection Probability: 48.19%
Watermark successfully removed or evaded for sample 1 at iteration 14!
Detection Probability: 48.19%
```

Whereelse, in the PSO attack, our PESQ score are pretty much consistent and was still able to evade detection.

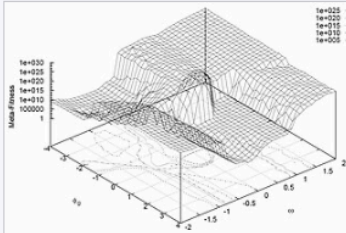
But overall, both attacks presented are essentially doing the same thing, adding noise (which looking back, we should have explored other audio alteration techniques, that could potentially make this PSO attack more unique than the query one) to the audio so that it can avoid detection. One is more structure in its way than the other.

With PSO, the parameters `(w, initial_epsilon, c1,c2)` played a huge part as it affects how the particle find the best swarm solution in the solution space. We came to a conclusion of the current parameters from trial-and-error. We have to emphasize how we might have not even utilize the power of PSO algorithm as it should, because without setting the random seed, the PSO and the normal querying is more or less the same.

[edit]

[11][12][13][14][15][16][17][18][19]

must be smaller than 1. The two other parameters can be then derived thanks to the constriction approach,^[16] or freely selected, but the analyses suggest convergence domains to constrain them. Typical values are in $[1, 3]$.



Performance landscape showing how a simple PSO variant performs in aggregate on several benchmark problems when varying two PSO parameters.

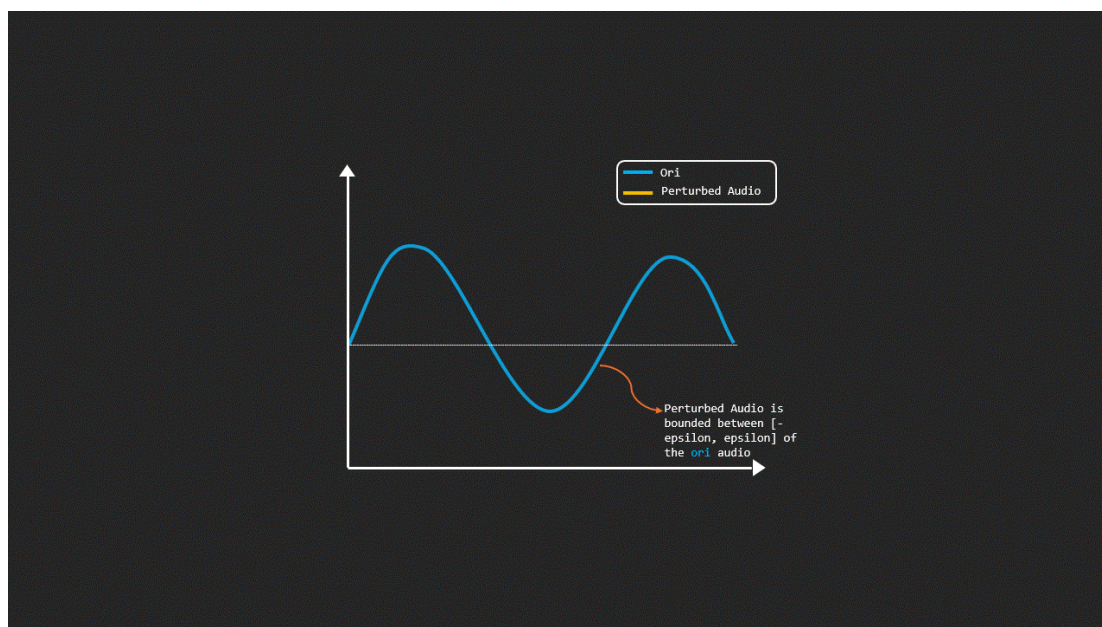
known as **meta-optimization**,^{[20][21][22][23]} or even fine-tuned during the optimization, e.g., by means of fuzzy logic.^{[24][25]}

Parameters have also been tuned for various optimization scenarios.^{[26][27]}

Here

PGD attack

Here



Projected Gradient Descent was first formally introduced in the following paper:-

TOWARDS DEEP LEARNING MODELS RESISTANT TO ADVERSARIAL ATTACKS

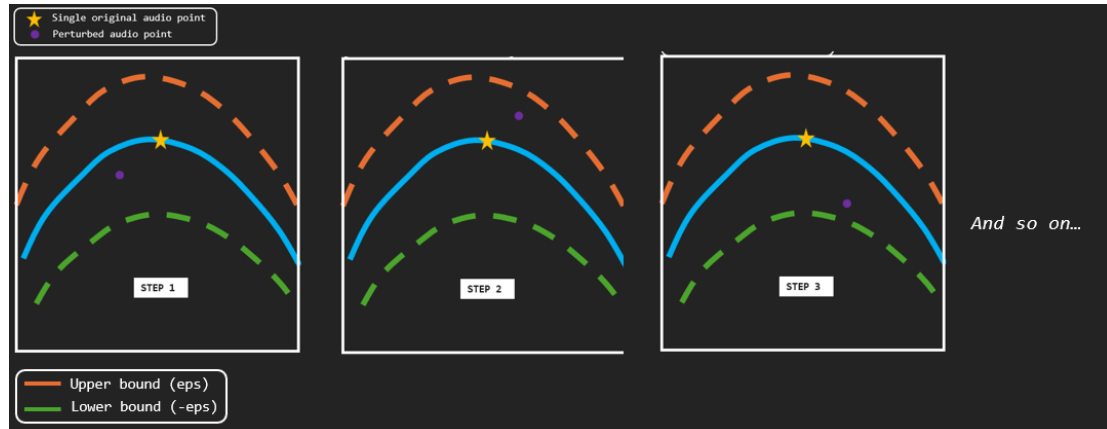
Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu*
 Department of Electrical Engineering and Computer Science
 Massachusetts Institute of Technology
 Cambridge, MA 02139, USA
 {madry, amakelov, ludwigs, tsipras, avladu}@mit.edu

ABSTRACT

Recent work has demonstrated that neural networks are vulnerable to adversarial examples, i.e., inputs that are almost indistinguishable from natural data and yet classified incorrectly by the network. To address this problem, we study the adversarial robustness of neural networks through the lens of robust optimization. This approach provides us with a broad and unifying view on much prior work on this topic. Its principled nature also enables us to identify methods for both training and attacking neural networks that are reliable and, in a certain sense, universal. In particular, they specify a concrete security guarantee that would protect against a well-defined class of adversaries. These methods let us train networks with significantly improved resistance to a wide range of adversarial attacks. They also suggest robustness against a *first-order adversary* as a natural security guarantee. We believe that robustness against such well-defined classes of adversaries is an important stepping stone towards fully resistant deep learning models.

The way PGD differs from the other black box attacks mentioned above, aside from the fact that they're different types of attack; PGD directly mimics

& alters the watermarked audio using the detector gradients to create a more targeted attack that exploits the model's weakness.



The `PGD` attack iteratively perturbs the audio signal (steps 1, 2, 3, and so on) while keeping the perturbations within the epsilon bounds (between the epsilon and -epsilon). The goal is to minimize the watermark detector's accuracy while maintaining audio quality.

Each step adjusts the signal slightly, moving the perturbed points away from the original points to confuse the detector. The process continues until either the detection accuracy falls below a threshold or the maximum number of steps is reached.

As the `PGD` attack is simply a technique, we need to define some criteria on what we want to achieve. In this case, it's the removal of the AudioSeal's watermark. The main methods in the `WatermarkDetectorWrapper` is specifically the `loss_audioseal`

The method computes the loss based on the detection probabilities returned by the watermark detector. For watermarked audio, it penalizes any positive watermark detection, helping the attack to reduce the watermark's presence across all frames of the audio.

Ultimately, we want to find a perturbation that removes the watermark.

`watermarked_probs = results[:, 1, :]` is the probability the audio contains the watermark. In each iteration, we are trying to reduce this a little by a little till it reaches our target, which we have set it to be below 30% in this case.

```
In [ ]: import time
```

```

class WatermarkDetectorWrapper():
    """
    class for the watermark detector model to use in pgd

    """

    def __init__(self, model, threshold, sample_rate=16000):
        """
        initializes the WatermarkDetectorWrapper with the model, detection
        and sample rate. assigns methods for calculating detection accuracy

        args:
            model: the watermark detector model to wrap.
            threshold: threshold for determining whether a watermark is detected
            sample_rate: the sample rate for audio processing. default is 16000
        """
        # this will be the detector model
        self.model = model
        # the threshold for watermark detection
        self.threshold = threshold
        self.sample_rate = sample_rate
        # set default methods to calculate detection accuracy and loss
        self.calculate_detection_accuracy = self.detection_accuracy_audio
        self.get_loss = self.loss_audioseal

    def loss_audioseal(self, audio):
        """
        computes the loss for the watermark detector. the loss is based on
        probability that the watermark is detected, penalizing any detected
        watermark

        args:
            audio: the input audio tensor to evaluate.

        returns:
            avg_penalty: the avg of the positive detection probabilities.
        """
        # get the detection probabilities from the model
        results, _ = self.model(audio, sample_rate=self.sample_rate)

        # extract probabilities for watermarked
        watermarked_probs = results[:, 1, :]

        # avg all penalties to get the total loss
        avg_penalty = torch.mean(watermarked_probs)

        return avg_penalty

    def detection_accuracy_audioseal(self, audio):
        """
        calculates the detection accuracy for a given audio sample.

        args:
            audio: the input audio tensor to evaluate.

```

```

    returns:
        result: watermark detector probability
    """
    # query the model to detect the watermark in the audio sample
    result, _ = self.model.detect_watermark(audio, sample_rate=self.s

    return result

```

```

In [ ]: # creating detector instance to be used in pgd
detector = WatermarkDetectorWrapper(
    model=watermark_detector,
    threshold=0.5)

```

During our exploration, we were exposed to more audio metrics, namely Signal-to-Noise Ratio (SNR) which measures the ratio of the original signal power to the noise (perturbation) power. This was surprisingly helpful to use during this stage, as it allowed us to monitor the audio's signal quality in real time.

The following is the [interpretation](#) of the SNR values:-

5 dB to 10 dB : is below the minimum level to establish a connection, due to the noise level being nearly indistinguishable from the desired signal (useful information).

10 dB to 15 dB : is the accepted minimum to establish an unreliable connection.

15 dB to 25 dB : is typically considered the minimally acceptable level to establish poor connectivity.

25 dB to 40 dB : is deemed to be good.

41 dB or higher : is considered to be excellent.

The SNR values correlates the PESQ values. The higher the better. Overall, anything above 15 decibels is good, but we try to aim for anything above 25 decibels to ensure a better adversarial audio quality.

$$\text{SNR} = 10 \times \log_{10} \left(\frac{\text{mean}(\text{watermarked_audio}^2)}{\text{mean}(\text{perturbation}^2)} \right)$$

```

In [ ]: def pgd_attack(detector, watermarked_audio, eps, alpha, steps, tau):
    """
    performs a projected gradient descent (pgd) attack on a watermarked a

```

```

args:
    detector: the watermark detector wrapped in a class, used to calc
    watermarked_audio: the input watermarked audio tensor.
    eps: the maximum perturbation allowed (epsilon).
    alpha: the step size for gradient updates.
    steps: the maximum number of pgd iterations.
    tau: the detection accuracy threshold for early stopping.

returns:
    best_adv_audio: the adversarially perturbed audio sample that min
"""
# record the start time of the attack
start_time = time.time()

# initialize the perturbation as a tensor of zeros, with gradient tra
perturbation = torch.zeros_like(watermarked_audio, requires_grad=True)

# detach the watermarked audio from any previous computation graph
watermarked_audio = watermarked_audio.detach()

# calculate the initial detection accuracy of the unperturbed audio
best_detection_accuracy = detector.calculate_detection_accuracy(water

# clone the original watermarked audio to keep track of the best adve
best_adv_audio = watermarked_audio.clone()

# create an optimizer to adjust the perturbation based on the loss
optimizer = torch.optim.Adam([perturbation], lr=alpha)

# pgd attack loop for the specified number of steps
for i in range(steps):

    # zero out gradients for the optimizer
    optimizer.zero_grad()

    # apply the current perturbation to the audio, ensuring it's with
    perturbed_audio = watermarked_audio + perturbation.clamp(-eps, ep

    # calculate the loss based on the perturbed audio
    loss = detector.get_loss(perturbed_audio)

    # backpropagate the loss to compute gradients
    loss.backward()

    # update the perturbation based on the computed gradients
    optimizer.step()

    # re-clip the perturbation to ensure it stays within the epsilon
    perturbation.data = perturbation.clamp(-eps, eps)

    # calculate the detection accuracy of the perturbed audio
    detection_accuracy = detector.calculate_detection_accuracy(pertur

```

```

# calculate the signal-to-noise ratio (snr) between the original
snr = 10 * torch.log10(torch.mean(watermarked_audio**2) / torch.m

# print the detection accuracy and snr for the current step
print(f"Step {i + 1}/{steps}: Loss = {loss.item():.3f}, Detection

# if the current detection accuracy is lower than the previous be
if detection_accuracy < best_detection_accuracy:
    best_detection_accuracy = detection_accuracy
    best_adv_audio = perturbed_audio.clone()

# stop early if the detection accuracy drops below the threshold
if best_detection_accuracy <= tau:
    print(f"Early stopping at step {i + 1}")
    break

# if the detection accuracy never drops below tau, report that the at
if best_detection_accuracy > tau:
    print(f'Attack failed, best detection accuracy is {best_detection

# print the total time taken for the attack
print(f"Attack finished in {time.time() - start_time:.2f} seconds")

# return the best adversarial audio found during the attack
return best_adv_audio

```

Similarly to our `PS0` attack, the hyperparameters that we had used were true trial & error with a couple of combinations. With the effectiveness of the `PGD` attack, we thought to reduce the `tau` (or the detector threshold) to see if we can lower the detection rate as much as we can, while still maintaining a high enough SNR score.

```

In [ ]: pgd_audio_list = []
        pgd_probabilities = []

for idx, (watermarked_audio, sample_rate) in enumerate(watermarked_audio_list):
    print(f"Processing sample {idx + 1}/{len(watermarked_audio_list)}...")

    # the pgd attack on the current watermarked audio sample
    adversarial_audio = pgd_attack(
        detector=detector,                # watermark detector
        watermarked_audio=watermarked_audio, # current watermarked audio
        eps=0.005,                        # perturbation strength
        alpha=0.1/255,                    # learning rate
        steps=40,                          # number of iterations
        tau=0.3                            # early stopping threshold
    )

    # storing adversarial audio and its probability
    pgd_audio_list.append((adversarial_audio, sample_rate))
    result, _ = watermark_detector.detect_watermark(adversarial_audio, sa

```

```
pgd_probabilities.append(result)
```

```
print(f"Sample {idx + 1} processed.")  
print(f"Detection Probability: {result*100:.2f}%")
```

Processing sample 1/1...

Step 1/40: Loss = 1.000, Detection accuracy = 100.00%, snr = 42.16

Step 2/40: Loss = 0.782, Detection accuracy = 77.10%, snr = 39.84

Step 3/40: Loss = 0.603, Detection accuracy = 60.15%, snr = 37.25

Step 4/40: Loss = 0.529, Detection accuracy = 53.20%, snr = 35.34

Step 5/40: Loss = 0.478, Detection accuracy = 47.84%, snr = 33.89

Step 6/40: Loss = 0.439, Detection accuracy = 44.34%, snr = 32.75

Step 7/40: Loss = 0.396, Detection accuracy = 39.94%, snr = 31.83

Step 8/40: Loss = 0.357, Detection accuracy = 35.84%, snr = 31.05

Step 9/40: Loss = 0.320, Detection accuracy = 32.35%, snr = 30.38

Step 10/40: Loss = 0.296, Detection accuracy = 29.63%, snr = 29.81

Early stopping at step 10

Attack finished in 50.13 seconds

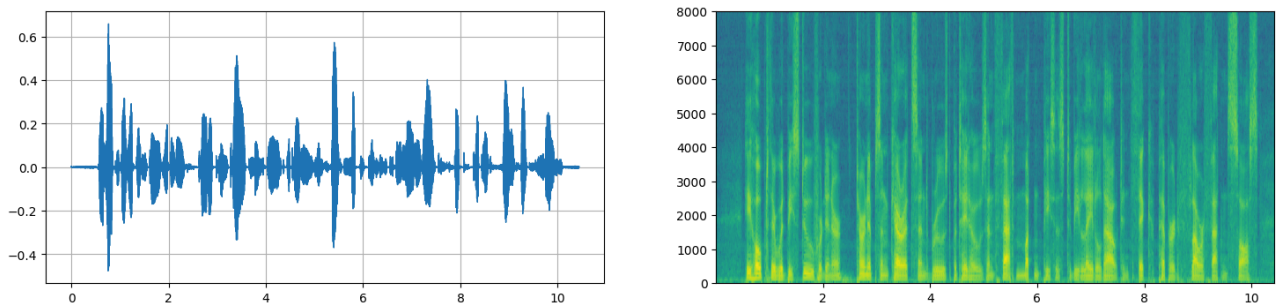
Sample 1 processed.

Detection Probability: 29.63%

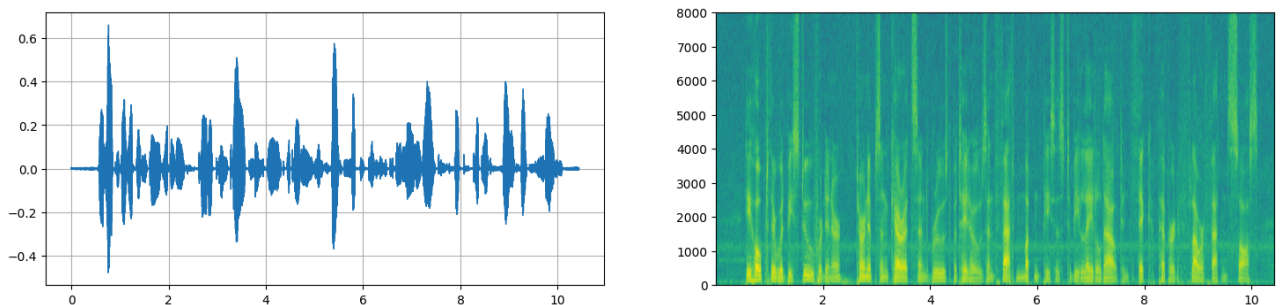
```
In [ ]: sample_pgd_audio = pgd_audio_list[0][0]
```

```
In [ ]: plot_waveform_and_spectrogram(after_watermark_audio, sample_rate, 'Waterma  
plot_waveform_and_spectrogram(sample_pgd_audio, sample_rate, 'Watermarked
```

Watermarked Audio: Before PGD Attack - waveform and spectrogram



Watermarked Audio: After PGD Attack - waveform and spectrogram



```
In [ ]: # calculating pesq score between the watermarked & after pgd attack  
pesq_score_pgd = pesq(after_watermark_audio.squeeze(), sample_pgd_audio.s  
print(f"PGD --> PESQ Score: {pesq_score_pgd:.2f}")
```

PGD --> PESQ Score: 3.71

```
In [ ]: play_audio(after_watermark_audio, sample_rate)
        play_audio(sample_pgd_audio, sample_rate)
```

Your browser does not support the audio element.

Your browser does not support the audio element.

The verdict?

```
In [ ]: # storing all relevant values from lists initialized in previous attacks

result_prob_dict = {'original(watermarked)': (watermark_probabilities[0],
        'query': (query_probabilities[0], pesq_score_query.item()),
        'pso': (pso_probabilities[0], pesq_score_pso.item()),
        'pgd': (pgd_probabilities[0], pesq_score_pgd.item()) )
        }

result_prob_dict
```

```
Out[ ]: {'original(watermarked)': (1.0, 4.498063087463379),
        'query': (0.47120267152786255, 1.9092580080032349),
        'pso': (0.3849484920501709, 1.7499735355377197),
        'pgd': (0.29626259207725525, 3.7124576568603516)}
```

```
In [ ]: # gathering values from above dict to plot
attack_types = list(result_prob_dict.keys())
probabilities = [value[0] for value in result_prob_dict.values()]
pesq_scores = [value[1] for value in result_prob_dict.values()]

# plotting results
bar_width = 0.35
index = range(len(attack_types))
fig, ax = plt.subplots(figsize=(10, 6))

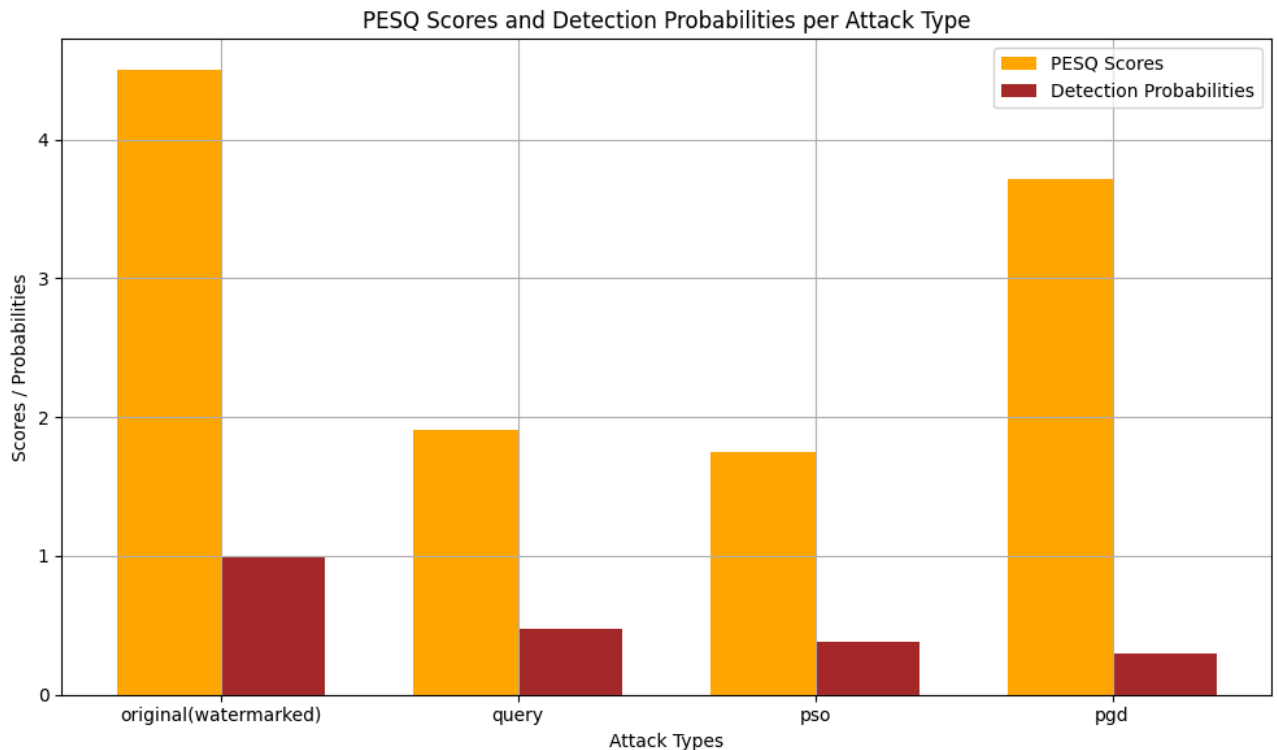
# PESQ scores
bar1 = ax.bar([i - bar_width / 2 for i in index], pesq_scores, bar_width,

# detection probabilities bars
bar2 = ax.bar([i + bar_width / 2 for i in index], probabilities, bar_widt

ax.set_xlabel('Attack Types')
ax.set_ylabel('Scores / Probabilities')
ax.set_title('PESQ Scores and Detection Probabilities per Attack Type')

ax.set_xticks(index)
ax.set_xticklabels(attack_types)
ax.legend()
ax.grid(True)
```

```
plt.tight_layout()
plt.show()
```



Indeed as expected from the attack graph in the paper we have introduced early on, whitebox attack are much more effective and also, subtle.

PGD is by far the best, with the PESQ & probability ranging from 3.7 - 4.17 on average.

PSO is a more sophisticated/guided method of adding noise to the watermarked data, and thru finetuning the parameters, we can get a consistent PESQ score.

Query was simply us adding a scaled noise to the attack until it bypass the detector. Depending on the run, if it can find the adversarial audio in early iteration, it can sometimes produce better results than PSO. In general the PESQ score for this attack is dependant on which iteration found the adversarial audio. But overall both Query & PSO are almost tied together.

However, instead of just focussing on the PESQ score, there could be some trade-off, ie: even though PSO has lower PESQ score, it also has a lower detection score. Although this observation might be pointless, as in the audio, we can actually hear the noise.

We should emphasize, in reality, it is more likely that one won't have access to the model, so even though PGD is much more subtle and effective, we should in general try to aim to develop sophisticated black-box attacks that can help bypass the detector.

As mentioned earlier, parameter tuning is very much essential for PS0, as it determines the quality of the audio (best solution found), with more time, black-box attacks can potentially help achieve better results than our current find.

Audio Results

Feel free to click here or play in the below cells.

```
In [ ]: print('+--+All audio results+--+')

print('\nOriginal Watermarked Audio')
play_audio(after_watermark_audio, sample_rate)

print('\nQuery Attack Audio')
play_audio(sample_query_audio, sample_rate)

print('\nPS0 Attack Audio')
play_audio(sample_pso_audio, sample_rate)

print('\nPGD Attack Audio')
play_audio(sample_pgd_audio, sample_rate)
```

+--+All audio results+--+

Original Watermarked Audio

Your browser does not support the audio element.

Query Attack Audio

Your browser does not support the audio element.

PS0 Attack Audio

Your browser does not support the audio element.

PGD Attack Audio

Your browser does not support the audio element.

End of Notebook

In **Milestone 4**, since it aims to maximise individual points, we referred to the [Prisoners Dilemma](#) in making our decision. In that scenario, the Nash Equilibrium for this specific dilemma was for both to confess, leading to them

both to stay in jail for a 5 years. But had they cooperated and shared a plan of defending, they would both only served 1 year, and that would be the best. There's a lot of what if's here, but in the assumption of both players are trustworthy and wont betray each other, cooperating and keeping quiet would have been a good, sensible deal for both.

That being said, we had cooperated to find out how to maximise points for the both of us to a certain extent. Since there wasnt explicitly a Light team for this topic, we will refer to the AudioSeal as the LightTeam, as the program is already very much hard to attack elegantly most of the time.

As we took upon an exploratory approach in finding ways to attack AudioSeal, we have some other attacks that we did not present in our Milestone 3 and had divided among us to be presented in our respective reports.

Reference:

- AudioSeal Paper & GitHub
 - Roman, R.S, Fernandez, P, Elsahar, H, & Défossez, A. (2024). *Proactive Detection of Voice Cloning with Localized Watermarking*. Hal.science, 235, 1-17. <https://hal.science/hal-04610152>
 - Source GitHub: [GitHub](#)
- Supplementary Resources:
 - Mun, H., Seo, S., Son, B., & Yun, J. (2022). *Black-Box Audio Adversarial Attack Using Particle Swarm Optimization*. IEEE Access, 10, 23532-23544. <https://doi.org/10.1109/access.2022.3152526>
 - Carlini, N., & Wagner, D. (2018, January 5). Audio Adversarial Examples: Targeted Attacks on Speech-to-Text. ArXiv.org. <https://doi.org/10.48550/arXiv.1801.01944>
 - Rix, A. W., Beerends, J. G., Hollier, M. P., & Hekstra, A. P. (2001). *Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs*. 2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No.01CH37221). <https://doi.org/10.1109/icassp.2001.941023>

- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2019). Towards Deep Learning Models Resistant to Adversarial Attacks. ArXiv:1706.06083 [Cs, Stat]. <https://arxiv.org/abs/1706.06083>
- FIT5215 (MUM). (2019). Tut_06b_AdversarialML_PyTorch. Google Collab. https://colab.research.google.com/drive/1miak_poNi7nuOy7UGhXPnIKpoPnUusHm?usp=sharing

Pseudocodes

Query Attack (blackbox)

Attack 1 Query Based

Input: Watermarked Audio, Perturbation Strength, Max Iterations, Watermark Detector, Threshold

Output: Perturbed Audio

```

1: Perturbed Audio  $\leftarrow$  Clone Watermarked Audio
2: for  $i$  in Max Iterations do:
3:    $\epsilon \leftarrow$  Generate random noise scaled by Perturbation Strength
4:   Perturbed Audio  $\leftarrow$  Perturbed Audio +  $\epsilon$ 
5:   Detector Probability  $\leftarrow$  Query Watermark Detector on Perturbed Audio
6:   if Detector Probability < Threshold then
       break
   endif
endfor
7: return Perturbed Audio

```

PSO Attack (blackbox)

Attack 2 Particle Swarmed Optimization

Input: Watermarked Audio, Watermark Detector, Swarm Size, Max Iterations, Initial Epsilon, Cognitive Factor 1 (C1), Cognitive Factor 2 (C2), Inertia Weight, Lower Termination Bound, Upper Termination Bound

Output: Perturbed Audio

```
1: Particles  $\leftarrow$  Initialize empty list
2: for i in Swarm Size do:
3:    $\varepsilon \leftarrow$  Generate random noise scaled by Initial Epsilon
4:   each Particle consist of
5:     Particle Position  $\leftarrow$  Watermarked Audio +  $\varepsilon$ 
6:     Particle Velocity  $\leftarrow$  Zero vector same size as Watermarked Audio
7:     Best Position  $\leftarrow$  Particle Position
8:     Best Fitness  $\leftarrow \infty$ 
9:   Add Particle to Particles
10: endfor
11: G. Best Fitness  $\leftarrow \infty$ 
12: G. Best Position  $\leftarrow$  None
13: for i in Max Iterations do:
14:    $\varepsilon \leftarrow$  Initial Epsilon (Perform Epsilon Decay)
15:   for Particle in Particles do:
16:     for j in Max Iterations do:
17:       Fitness  $\leftarrow$  Evaluate Fitness on Particle Position
18:       if Fitness < Best Fitness then
19:         Best Fitness  $\leftarrow$  Fitness
20:         Best Position  $\leftarrow$  Particle Position
21:       endif
22:       if Fitness < G. Best Fitness then
23:         G. Best Fitness  $\leftarrow$  Fitness
24:         G. Best Position  $\leftarrow$  Particle Position
25:       endif
26:     endfor
27:     for Particle in Particles do:
28:       R1, R2  $\leftarrow$  Random Factors
29:       Particle Velocity  $\leftarrow$  Inertia Weight * Particle Velocity + C1 * R1 * (Best Position - Particle Position) + C2 * R2 * (G. Best Position - Particle Position)
30:       Particle Position  $\leftarrow$  Particle Position + Particle Velocity
31:     endfor
32:   if G. Best Fitness  $\geq$  Lower Termination Bound and G. Best Fitness  $\leq$  Upper Termination Bound then
33:     break
34:   endif
35: endfor
36: Perturbed Audio  $\leftarrow$  G. Best Position
37: return Perturbed Audio
```

PGD Attack (whitebox)

Attack 3 Projected Gradient Descent (*Watermark Removal*)

Input: Watermarked Audio, Perturbation Strength (ϵ), Learning Rate (α), Max Iterations, Watermark Detector, Accuracy Threshold (τ)

Output: Best Perturbed Audio

```
1: Perturbation  $\leftarrow$  Initialize zero tensor of same shape as
   Watermarked Audio
2: Best Detection Accuracy  $\leftarrow$  Calculate Detection Accuracy
   of Watermarked Audio
3: Best Perturbed Audio  $\leftarrow$  Clone Watermarked Audio
4: for  $i$  in Max Iterations do:
5:     Clear gradients of optimizer
6:     Perturbed Audio  $\leftarrow$  Watermarked Audio + Clamp
       (Perturbation,  $-\epsilon$ ,  $\epsilon$ )
7:     Loss  $\leftarrow$  Get loss of Perturbed Audio
8:     Backpropagate Loss
9:     Update Perturbation using optimizer
10:    Clamp Perturbation within  $[-\epsilon, \epsilon]$ 
11:    Current Detection Accuracy  $\leftarrow$  Calculate
       Detection Accuracy of Perturbed Audio
12:    SNR  $\leftarrow$  Calculate Signal-to-Noise Ratio of
       Watermarked Audio & Perturbation
13:    if Current Detection Accuracy  $<$  Best
       Detection Accuracy then:
14:        Best Detection Accuracy  $\leftarrow$  Current
           Detection Accuracy
15:        Best Perturbed Audio  $\leftarrow$  Clone Perturbed
           Audio
16:    endif
17:    if Best Detection Accuracy  $\leq$  Accuracy Threshold
       then:
18:        break
19:    endif
20: endfor
21: return Best Perturbed Audio
```

In []:

In []:

In []: